

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/312057617>

Randomness in Neural Networks: An Overview

Article in *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* · January 2017

DOI: 10.1002/widm.1200

CITATIONS

343

READS

13,087

2 authors:



[Simone Scardapane](#)

Sapienza University of Rome

193 PUBLICATIONS 4,580 CITATIONS

[SEE PROFILE](#)



[Dianhui Wang](#)

La Trobe University

212 PUBLICATIONS 5,708 CITATIONS

[SEE PROFILE](#)



Randomness in neural networks: an overview

Simone Scardapane¹ and Dianhui Wang^{2*}

Neural networks, as powerful tools for data mining and knowledge engineering, can learn from data to build feature-based classifiers and nonlinear predictive models. Training neural networks involves the optimization of nonconvex objective functions, and usually, the learning process is costly and infeasible for applications associated with data streams. A possible, albeit counterintuitive, alternative is to randomly assign a subset of the networks' weights so that the resulting optimization task can be formulated as a linear least-squares problem. This methodology can be applied to both feedforward and recurrent networks, and similar techniques can be used to approximate kernel functions. Many experimental results indicate that such randomized models can reach sound performance compared to fully adaptable ones, with a number of favorable benefits, including (1) simplicity of implementation, (2) faster learning with less intervention from human beings, and (3) possibility of leveraging overall linear regression and classification algorithms (e.g., ℓ_1 norm minimization for obtaining sparse formulations). This class of neural networks attractive and valuable to the data mining community, particularly for handling large scale data mining in real-time. However, the literature in the field is extremely vast and fragmented, with many results being reintroduced multiple times under different names. This overview aims to provide a self-contained, uniform introduction to the different ways in which randomization can be applied to the design of neural networks and kernel functions. A clear exposition of the basic framework underlying all these approaches helps to clarify innovative lines of research, open problems, and most importantly, foster the exchanges of well-known results throughout different communities. © 2017 John Wiley & Sons, Ltd

How to cite this article:

WIREs Data Mining Knowl Discov 2017, 7:e1200. doi: 10.1002/widm.1200

INTRODUCTION

Training neural networks (NNs) is a process involving the optimization of a highly nonconvex landscape, where local minima and saddle points are widespread and for which the computation of gradients pose fundamental challenges.¹ Alternative kernel-based approaches result instead in well-posed

convex problems, one of the key reasons for their success in the last decade.² Kernel methods, however, struggle to scale efficiently to large datasets since, among other things, they require the computation of pairwise kernel values over the entire dataset. In addition to this, it is known that optimal testing accuracy achieved by recent deep generations of NNs³ is only one among many conflicting objectives in data mining applications, which—depending on the problem at hand—might include a short training time, the possibility of hardware implementation, model complexity, stability of the software libraries, robustness to noise, and so on.⁴ This is why, even in our era of deep learning and big data, it remains essential to understand the limits and the potentialities of simpler (i.e., single-layer) neural architectures,

*Correspondence to: dh.wang@latrobe.edu.au

¹Department of Information Engineering, Electronics and Telecommunications (DIET), Sapienza University of Rome, Rome, Italy

²Department of Computer Science and Information Technology, La Trobe University, Melbourne, Australia

Conflict of interest: The authors have declared no conflicts of interest for this article.

allowing a satisfactory level of solution to be reached within one-hundredth (or even one-millionth) of the time required by larger, more complex models while at the same time possessing strong optimality guarantees in their optimization process, which can be customized and implemented easily.

One successful approach in this sense is the use of ‘*randomization*,’ that is, the stochastic assignment of a subset of the NNs’ weights in order to derive a simpler (often linear) optimization problem to solve. This idea has been applied countless times over the years, and it has resulted in three broad families of NN models that we classify as follows: (1) feedforward networks with random weights (RW-FFN),⁵ (2) recurrent NNs with random weights (i.e., reservoir computing, RC),⁶ and (3) randomized kernel approximations.⁷ Although the literature on these three methods is generally kept separate, they all share two fundamental ideas that contribute to their success. First, in all three cases, randomization is used to define a (generally data independent) feature map, which transforms the input into a highly dimensional space where learning is supposed to be simpler. Secondly, the resulting optimization problem is cast as a standard linear least-squares, which is by far the simplest, most studied and scalable learning procedure to date. In a sense, many of these methods are direct descendants of the seminal work of Cover on the linear separability of patterns,⁸ and they are elegantly summarized in the recent quote from Rahimi and Recht, that “*randomization is [...] cheaper than optimization.*”⁹

Both of the aforementioned aspects can help to explain the remarkable successes obtained by randomized NN models in realistic data mining problems. State-of-the-art libraries for linear regression are able to handle very large feature spaces, where the probability of finding a hyperplane with good generalization capabilities can be extremely high. In addition to this, drawing inspiration from the vast literature on linear regression (which goes as far back as Gauss himself¹⁰), it is possible to customize the training procedure in a straightforward way with a vast number of dedicated algorithms. As an example, sparsity on the adaptable parameters can be obtained immediately by incorporating a sparse penalty and solving the resulting linear problem in all families.¹¹ At the same time, the literature on these topics is vast and fragmented so that equivalent ideas are reintroduced time and again, and it becomes difficult to appreciate the fundamental unity (both theoretical and practical) underlying all these methods. Going back to our previous example, sparse linear regression has been derived

independently in all three areas, sometimes more than once in each case.^{12–14}

Our aim in this overview is to provide a concise, self-contained, and uniform introduction to these three families of methods. Apart from providing a single entry point into a growing (and decades-spanning) body of literature, clearly stating the basic framework underlying them can help to clarify new lines of research and, more importantly, to foster the exchanges of well-known results throughout different communities. Two broad themes emerge from our treatment, which are interesting to briefly discuss here. First and foremost, there is a clear indication that random weight assignment is not just a ‘cheap trick’ to solve an otherwise difficult problem. On the contrary, the algorithms discussed here have continuously achieved remarkable accuracy, sometimes comparable to deep architectures,¹⁵ in a large number of applications, ranging from short-term forecasting to image recognition, biomedical classification, and many others. This apparently counterintuitive insight warrants more attention as its reasons can point to a deeper understanding of the working behavior of NNs themselves. As discussed in the work of Saxe et al.,¹⁶ for example, it appears that “*a sizeable component of a system’s performance can come from the intrinsic properties of the architecture*” so that a good architecture with a subset of weights assigned randomly can easily outperform a poorer architecture with finely tuned weights. This also explains why research in this field, far from withering away, has continued to grow relentlessly as available computational power increased.

The second important aspect highlighted here is that randomized methods can lead the way to some intriguing theoretical analyses on their behavior in terms of accuracy, dynamics, and memory. As an example, RW-FFNs are shown to have error bounds comparable to that of networks with completely adaptable parameters. One additional aim of our overview is to summarize some of the most fundamental properties evidenced by these studies along with the open problems and challenges that need to be resolved.

Structure of This Overview

Following our categorization of randomized NN models, this overview is organized into three parts. The first section, ‘*Feedforward networks with random weights*,’ introduces the simplest randomized NN model, the RW-FFN. The content of this section is the foundation for the remainder of the paper; in fact, all training methods and most

approximation properties that we describe are common to *all* families in the overview. The following sections describe the two remaining families of models, namely random features for kernel methods and the RC framework. In both cases, we first show how the learning problem is equivalent to that of RW-FNN before focusing on some aspects that are specific to each model, such as the theoretical analysis of the reservoir in RC networks. Finally, the last section draws some general conclusions from our overview.

FEEDFORWARD NETWORKS WITH RANDOM WEIGHTS

Network Architecture

The basic model is defined for a generic input $\mathbf{x} \in \mathbb{R}^d$ by linearly combining B nonlinear transformations of the original input, where B is chosen by the user (see Figure 1):

$$f(\mathbf{x}) = \sum_{m=1}^B \beta_m h_m(\mathbf{x}; \mathbf{w}_m) = \boldsymbol{\beta}^T \mathbf{h}(\mathbf{x}; \mathbf{w}_1, \dots, \mathbf{w}_B), \quad (1)$$

where the m th transformation is parameterized by the vector $\mathbf{w}_m$. In Eq. (1), the weights $\mathbf{w}_m$ are drawn randomly from a predefined probability distribution, while the linear coefficients $\boldsymbol{\beta}$ are adapted based on some training data. Although the model is derived for a single output only, everything that follows can be extended straightforwardly to the case of multiple outputs. For readability, in the following, we drop the dependence of h_m with respect to $\mathbf{w}_m$.

There are two basic families of methods falling under this scheme, depending on the type of hidden functions $h_m(\cdot)$. In the *additive* case, each function applies a predefined nonlinearity on a random linear combination of its input vector:

$$h_m(\mathbf{x}) = g(\mathbf{a}_m^T \mathbf{x} + b_m), \quad (2)$$

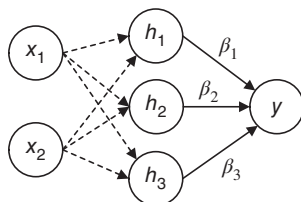


FIGURE 1 | An RW-FNN architecture with two inputs, three hidden functions, and one output. Fixed connections are shown as dashed lines, whilst trainable connections are fixed lines.

with $\mathbf{w}_m = [\mathbf{a}_m^T b_m]^T$. A typical choice for $g(\cdot)$ in this case is the sigmoid function:

$$h_m(\mathbf{x}) = \frac{1}{1 + \exp\{-\left(\mathbf{a}_m^T \mathbf{x} + b_m\right)\}}. \quad (3)$$

If we stack, row-wise, all the vectors $\mathbf{a}_m$ and scalars b_m into a matrix $\mathbf{A}$ and vector $\mathbf{b}$, respectively, the first layer can be written compactly as $g(\mathbf{A}\mathbf{x} + \mathbf{b})$, where $g(\cdot)$ now operates element-wise. The linear projection $\mathbf{A}\mathbf{x}$ is loosely connected to the well-known method of random projections (RP),^{17,18} a fact that might engender confusion. In RP, the matrix $\mathbf{A}$ is also randomly generated, but the aim is to reduce the dimensionality of the data while simultaneously approximately preserving the Euclidean distances among the projected points. Since the aims are different, there are two broad differences with respect to the RW-FNN that have to be kept in mind: (1) in order to have sufficiently expressive power, in our case, B is typically much larger than the number of input dimensions (even by one order of magnitude), and (2) the linear transformation $\mathbf{A}$ is not required to preserve distances.¹⁷ In fact, it is standard practice to randomly sample its values from a predefined distribution (e.g., the uniform distribution in $[-1, +1]$), which might not respect the theory of RP.

In the second family of methods, each function is chosen instead as a radial basis function (RBF), typically of Gaussian shape:

$$h_m(\mathbf{x}) = \exp\left\{-\alpha_m \|\mathbf{x} - \mathbf{c}_m\|_2^2\right\}, \quad (4)$$

with $\mathbf{w}_m = [\mathbf{c}_m^T \alpha_m]^T$, and $\|\cdot\|_2$ is the standard Euclidean norm. In this case, the network can be seen as an RBF network with randomly chosen centers $\mathbf{c}_m$ and scaling factors α_m , corresponding to a superposition of B Gaussian distributions centered at random places in the input space.¹⁹ Alternatively, less common choices for $g(\cdot)$ are also possible, including multiplicative univariate functions,²⁰ convolutive filters with random weights (for 2D inputs),²¹ and others. Random features for approximating kernels, discussed in the next section, also fit the basic model in Eq. (1). However, they derive from an intermediate kernel representation, and for this reason, we discuss them separately.

Training the Network

Given a model in Eq. (1), we need efficient algorithms to adapt its parameters. Again, we stress that the content of this section applies directly to the

training of *all* models considered in the next sections. Suppose we are provided with a set of N samples of the desired function to be approximated, that is, $S = \{\mathbf{x}_i, y_i\}$, $i = 1, 2, \dots, N$. Denoted by $\mathbf{H}$ the *hidden* matrix:

$$\mathbf{H} = \begin{pmatrix} h_1(\mathbf{x}_1) & \cdots & h_B(\mathbf{x}_1) \\ \vdots & \ddots & \vdots \\ h_1(\mathbf{x}_N) & \cdots & h_B(\mathbf{x}_N) \end{pmatrix}. \quad (5)$$

Choosing the optimal $\boldsymbol{\beta}$ can obviously be formulated as a standard regularized least-squares problem:

$$\boldsymbol{\beta}^* = \arg \min_{\boldsymbol{\beta} \in \mathbb{R}^B} \left\{ \frac{1}{2} \|\mathbf{H}\boldsymbol{\beta} - \mathbf{y}\|_2^2 + \frac{\lambda}{2} \|\boldsymbol{\beta}\|_2^2 \right\}, \quad (6)$$

where $\mathbf{y}$ is a column vector containing all the desired outputs, and the user-defined scalar λ is weighting the regularization term. The problem in Eq. (6) is strictly convex, so its solution can be found by setting its gradient to 0, that is:

$$\boldsymbol{\beta}^* = (\mathbf{H}^T \mathbf{H} + \lambda \mathbf{I})^{-1} \mathbf{H}^T \mathbf{y}, \quad (7)$$

where $\mathbf{I}$ is the identity matrix of suitable dimensionality. The fact that the training of the model (more generally, of all the models described in this overview) can be performed in a single analytical step that is implemented efficiently in most linear algebra libraries is the major advantage of these methods. The computational complexity of least-squares in this case is mostly influenced by the $B \times B$ matrix inversion in Eq. (7). In cases where $N \ll B$, standard algebra also allows for an alternative formulation, where the matrix to be inverted is reduced to dimensionality $N \times N$:

$$\boldsymbol{\beta}^* = \mathbf{H}^T (\mathbf{H} \mathbf{H}^T + \lambda \mathbf{I})^{-1} \mathbf{y}. \quad (8)$$

For large-scale problems, parallel versions of Eq. (6) exist in most parallel computing frameworks. For example, matrix–matrix and matrix–vector products in Eq. (6) can be computed in subgroups using a MapReduce environment²² and summed up before inversion in a reducer node. A streaming parallel implementation is also available on the popular Apache Spark library,^a and matrix decompositions for computing the Moore–Penrose inverse $(\mathbf{H}^T \mathbf{H} + \lambda \mathbf{I})^{-1} \mathbf{H}^T$ can be parallelized using graphical processing unit (GPU) or hybrid central processing unit (CPU)/GPU clusters.²³

Additionally, the problem can be addressed using convex optimization routines such as the

conjugate gradient⁵ or primal–dual strategies,²⁴ which in the case of quadratic problems such as Eq. (6) possess very fast convergence guarantees. The problem can also be solved efficiently in the case of streaming data (with possible time-varying characteristics) by the use of the well-known recursive least-squares algorithm.²⁵ For binary classification problems (i.e., when the output can only take binary values), the simplest approach is to combine the least-squares criterion in Eq. (6) with the subsequent binarization of the RW-FNN outputs. More efficient loss functions (e.g., hinge loss) are possible but do not admit a closed-form solution anymore; a review on recent progress for large-scale linear classification is given by Yuan et al.²⁶

While this is enough to implement a working RW-FNN in most situations, it only scratches the surface of all possible modifications for its training algorithm, which can leverage in a straightforward fashion over all available literature on linear regression/classification. Some of these have also been explicitly considered in the case of RW-FNN. As a representative example, ℓ_1 minimization can be used to achieve *sparse* output layers¹²:

$$\boldsymbol{\beta}^* = \arg \min_{\boldsymbol{\beta} \in \mathbb{R}^B} \left\{ \frac{1}{2} \|\mathbf{H}\boldsymbol{\beta} - \mathbf{y}\|_2^2 + \lambda \|\boldsymbol{\beta}\|_1 \right\}, \quad (9)$$

where $\|\boldsymbol{\beta}\|_1 = \sum_{m=1}^B |\beta_m|$. This is the much-celebrated LASSO problem, for which extremely efficient optimization procedures are available.¹¹ Other possible variations are the absolute penalization of errors for the robust handling of noise and outliers,²⁷ ensembling for increasing accuracy and lowering variance,²⁸ and so on. However, a complete overview of the least-squares theory is beyond the scope of this paper, and we refer the reader to standard textbooks on the topic.²⁹

Historical Overview on RW-FNN Models

The basic idea represented by Eq. (1) is simple, yet it provides a very efficient nonlinear model. For this reason, similar ideas have been proposed many times, in different forms, over the last several decades. Here, we provide a brief historical overview of some representative works.

The Perceptron

A historical predecessor to RW-FNNs can be found in the perceptron, one of the first adaptable connectionist models explored by Frank Rosenblatt from 1957 onward.³⁰ In the perceptron, a sparse, random

layer of weights connects the sensory perceptions to an adaptable threshold layer (see, e.g., Fig. 13 in Widrow and Lehr³¹). This architecture was designed to work only for binary/ternary activations, and most importantly, its original training rule does not converge in the case of patterns that are not linearly separable: *“While it will eventually visit an optimal set of weights, it will not converge to any set of weights. Even worse, the algorithm can go from an optimal set of weights to a worst-possible set in one iteration, regardless of how many iterations have been taken previously.”*³² This aspect was at the base of the fundamental historical controversy surrounding this model.³³ Later modifications to the perceptron came closer to the RW-FNN as described previously, most notably the variation of the ‘pocket algorithm’ proposed by Gallant and Smith in 1987.³² However, they failed to gain additional popularity at the time.

Random Vector Functional-Link Networks

In the 1990s, RW-FNNs were introduced in their current form by a research group led by Y.-H. Pao under the name random vector functional-link (RVFL) networks.^{5,34,35} The only difference with respect to our treatment is the straightforward inclusion of direct links from the input to the output layer, which does not change the successive mathematical derivation. Apart from proving the universal property of RVFL networks (see the next section), they showed their usefulness on a number of real-world problems, particularly including handwritten script recognition.³⁶ To this end, they also investigated how RVFL networks can be customized to offer a practical tool for density estimation.³⁷ RVFL networks have gained increasing popularity in the last few years, and numerous additional applications were explored, which are briefly summarized in Section 1 of Martinez-Villena et al.³⁸ An extensive evaluation of different RVFL variants is given instead in Zhang and Suganthan,³⁹ particularly showing significant improvements by the inclusion of the input/output links. RVFL networks have also been explored in semisupervised scenarios, wherein only a part of the training data is labeled,⁴⁰ and more recently in a multilayered configuration using a variation of the autoencoder network.⁴¹

RBF Networks with Random Centers

In their RBF form (4), RW-FNNs were already envisioned in the seminal work of Lowe and Broomhead on RBF networks,¹⁹ in which the random sampling of centers from the training data is a good alternative to more sophisticated clustering procedures. Lowe and Broomhead, in a footnote, hinted at the

possibility of choosing RBF centers that do not correspond to any of the training points. This strategy was followed by a few authors, including Looney,⁴² but never became mainstream because of the possibility that random centers might not represent the actual distribution of points in the training dataset at hand.

Additional Considerations

In a 1992 conference publication, Schmidt et al. presented a network that closely resembled the additive version of the RW-FNN.⁴³ The network, however, had a completely different significance. Far from being a practical learning algorithm, it was used as a paradoxical criterion to explore some bizarre aspects of NNs, namely the possibility of obtaining high accuracies even when the number of parameters far exceeded the available training data, a result which goes against intuitions from learning theory. In this sense, the authors’ conclusion that *“the parameters found in the hidden layer do not add very much functionality to the classifier”* was not a justification for randomization but must be read in a wider line of skepticism toward NNs themselves, whose application is sometimes viewed as a black-box approach without a ‘real’ understanding of the problem.⁴⁴ The significance of the work of Pao and colleagues can be found exactly in their acceptance that the RW-FNN is not only efficient but also *practical*, and this is clear insofar as the RVFL is presented as a *“computational approach compatible with our goal of devising a general-purpose artificial neural-net computer.”*³⁴ This comment is echoed in a discussion by Widrow,⁴⁵ who replied to some general criticisms after having introduced an algorithm very similar to the RW-FNN, termed “NoProp algorithm”⁴⁶: *“we [...] have independently discovered that it is not necessary to train the hidden layers of a multi-layer NN. Training the output layer will be sufficient for many applications.”* It should be pointed out that some statements on the full-rank property of the hidden output matrix in this work are not entirely justified, and some further research is to be expected.

Before concluding this section, we remark that some interesting work has also been conducted to justify the RW-FNN approach from a biological perspective, as shown in the following quote by Tapson and van Schaik: *“we are starting to see some evidence from neurophysiology that structures embodying the [random weights] principle may exist in the brain. For example, recent work [...] shows that complex rule-based tasks require both sensory stimuli and internal representation of states; and that a significant number of random connections placed between input sources and a hidden interlayer, and*

random recurrent connections between interlayer neurons, are necessary for optimal performance. [...] these interlayer neurons [are] equivalent to increasing the dimensionality of the state representation.”⁴⁷ We will return to this point later on.

Universal Approximation Properties

The basic theoretical result on RW-FNNs was proven in 1995 by Igel'nik and Pao,³⁵ with later corrections made by Li et al.⁴⁸ They showed that, for sufficiently smooth functions, and under a proper choice of $h_m(\cdot)$ and the range from which parameters are extracted, the network possesses a universal approximation capability. Particularly, the order of approximation error is $\mathcal{O}(C/\sqrt{B})$, where the constant C is independent of B . Thus, a desired level of accuracy can, in principle, be reached by increasing the number of hidden nodes. The result is extremely significant because this error bound is, in fact, comparable to that of a network with fully adaptable connections. It also goes against the worst-case bound $\mathcal{O}(d^{-1}/B^{1/d})$ presented in the seminal work by Barron for a generic linear combination of fixed functions.⁴⁹ This apparent discrepancy is explored by Gorban et al.,⁵⁰ who underline the probabilistic nature of the result by Igel'nik and Pao: “[...] *there is always a non-zero probability of an unlucky draw from the probability distribution which will require re-initialization at some stage of approximation. Furthermore, [...] this rapid convergence of order $1/\sqrt{B}$ is assured only up to a given and fixed tolerance.*” From this, they draw the following warning: “*These features of randomized approximators should thus be considered with special care in applications.*” In this sense, how to properly choose a range of parameters and a sampling distribution remains an open problem in the literature so that the advantages given by RW-FNN approaches are paid by the possibility of sampling a network with extremely poor accuracy. Similar statements were made in a recent overview by Principe and Chen⁵¹: “[*random-weights models*] still suffer from design choices, translated in free parameters, which are difficult to set optimally with the current mathematical framework, so practically they involve many trials and cross validation to find a good projection space, on top of the selection of the number of hidden [processing elements] and the nonlinear functions.” It should be pointed out that the theoretical work established by Igel'nik and Pao³⁵ does not address the learning algorithm or implementation issues. From an algorithmic perspective, Li and Wang⁵² investigated the universal

approximation property of RVFL networks and reported a result on the infeasibility of such a learner model if it is built incrementally with random input weights and biases extracted from a fixed scope, and its convergence rate satisfies some certain condition. These limitations should always be kept in mind.

Recently, more refined theoretical bounds have been obtained by a number of authors in the kernel approximation literature, most notably Rahimi and Recht in 2008–2009 (who ironically referred to the model as ‘sums of random kitchen sinks’)^{9,53} and Rudi et al. in 2016.⁵⁴ The latter work also puts forth the idea that randomization can serve as a form of ‘computational regularization’ during the training process. The theory behind these works is the subject of the next section.

RANDOM FEATURES FOR KERNEL APPROXIMATION

Network Architecture

The second family of methods that we investigate is a special case of the model in Eq. (1), but its understanding is built on the notion of kernel functions. Due to this, we first provide a brief introduction to kernel methods before analyzing their drawbacks as motivation for introducing random features for their approximation.

A Brief Primer on Kernel Methods

A common alternative to the stochastic assignment of weights in Eq. (1) is to consider a *deterministic* B -dimensional feature mapping $\mathbf{h}(\cdot)^b$, for example, by the use of polynomial functions of the input. In some cases, however, this strategy requires the use of an intractable number of bases that can grow cubically or exponentially in the number of inputs. Kernel functions, as a way to circumvent this problem, were popularized by Cortes and Vapnik⁵⁵ and became the foundation for a wide array of algorithms in machine learning.² Informally, it can be shown that for a large number of mappings $\mathbf{h}(\cdot)$, there exists a function $\kappa : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ such that:

$$\mathcal{K}(\mathbf{x}_1, \mathbf{x}_2) = \mathbf{h}(\mathbf{x}_1)^T \mathbf{h}(\mathbf{x}_2). \quad (10)$$

The function $\mathcal{K}(\cdot, \cdot)$ is called a kernel, and it allows for an efficient evaluation of the dot product in the transformed space without the need for computing the mapping itself. One basic result, called the representer’s theorem, shows that the solution to a large number of regularized, supervised learning problems when using kernels can be expressed as:

$$f^*(\mathbf{x}) = \sum_{i=1}^N \alpha_i \mathcal{K}(\mathbf{x}, \mathbf{x}_i) \quad (11)$$

so that an optimization over a possibly infinite-dimensional space of functions is reduced to a finite-dimensional problem over the coefficients $\{\alpha_i\}_{i=1}^N$. Similarly, most problems in this case need to work with the so-called *kernel matrix* $\mathbf{K} \in \mathbb{R}^{N \times N}$, defined as:

$$K_{ij} := \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j). \quad (12)$$

As an example, the optimal solution $\boldsymbol{\alpha}^* = [\alpha_1, \dots, \alpha_N]^T$ to the kernel-based version of Eq. (6) (called kernel ridge regression) is written as:

$$\boldsymbol{\alpha}^* = (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y}. \quad (13)$$

Drawbacks of Kernel Methods for Large-Scale Problems

The previous discussion, albeit short, already clarifies most of the drawbacks encountered during the application of any kernel method to a large-scale application:

1. First and foremost, the computation of $\mathbf{K}$ scales quadratically with N , which might be prohibitive both in time and in memory, even when using sophisticated linear algebra libraries.
2. Secondly, the computational complexity of solving the optimization problems involved in computing the optimal weight vector $\boldsymbol{\alpha}^*$ are between quadratic and cubic in N , even for methods that are provably sparse in $\boldsymbol{\alpha}^*$ (such as support vector machines).
3. A third, less visible problem appears in online applications, wherein the number of terms in Eq. (11) grows unbounded. This problem is particularly daunting in adaptive filtering applications that exhibit stringent real-time requirements.⁵⁶
4. Finally, Eq. (11) is defined in terms of the original data, thus also being problematic for applications with privacy concerns, which are common in big data scenarios.⁵⁷

Kernel Approximation Techniques

Over the past years, a large number of approximation techniques have been developed to overcome the previously mentioned drawbacks in order to make

kernel methods affordable even in large-scale situations. These methods use randomization to approximate several aspects of the training process, for example, by appropriately sampling elements of the kernel matrix,⁵⁸ computing a low-rank approximation of $\mathbf{K}$,⁵⁹ or designing a stochastic approximation to the kernel function $\mathcal{K}(\cdot, \cdot)$ itself.⁷

The most well-known example of the second class is the Nyström method, originally proposed by Williams and Seeger⁶⁰ and refined by Drineas and Mahoney.⁵⁹ The Nyström method is a two-step process, which can be simplified as follows: (1) we first build a matrix $\mathbf{G}$ by randomly sampling $M \ll N$ columns of $\mathbf{K}$ according to a predefined strategy and scaling them appropriately; (2) then, we compute a low-rank approximation of $\mathbf{K}$ by $\hat{\mathbf{K}} = \mathbf{G} \mathbf{K}_k^+ \mathbf{G}^T$, where $\mathbf{K}_k^+$ is the best k -rank approximation of the square part of $\mathbf{K}$ corresponding to the indexes sampled in the step before (also scaled appropriately, see Drineas and Mahoney⁵⁹ for more details). While the Nyström method makes use of a randomization step, it does not result in a proper linear problem, and for this reason, we do not discuss it further here (although we will use it for comparison later on).

The third class of algorithms is instead interested in finding a good, low-dimensional approximation of the kernel function so as to obtain a feature mapping in the form (1), which is cheap to compute in real time. The idea was originally introduced by Rahimi and Recht for a particular class of kernels called shift-invariant⁷ and later generalized by a number of authors for other classes, including group-invariant kernels,⁶¹ γ -homogeneous kernels,⁶² and dot-product kernels,⁶³ among others. The idea can be described in a general form as follows. Denote by ν a random variable with probability density $P(\nu)$, and suppose the kernel can be expressed as an expectation form:

$$\mathcal{K}(\mathbf{x}_1, \mathbf{x}_2) = \mathbb{E}_{\nu \sim P(\nu)} [\phi_\nu(\mathbf{x}_1) \phi_\nu(\mathbf{x}_2)], \quad (14)$$

where $\phi_\nu(\cdot) : \mathbb{R}^d \rightarrow \mathbb{R}$ are a family of functions indexed by the random variable ν , and $\nu \sim P(\nu)$ denotes that the expectation is taken with respect to $P(\nu)$. Then, we can construct a low-dimensional approximation $\tilde{\mathcal{K}}$ of $\mathcal{K}$ with a Monte Carlo approach by randomly sampling B values $\{\nu_1, \dots, \nu_B\}$ from $P(\nu)$ to approximate Eq. (14):

$$\tilde{\mathcal{K}}(\mathbf{x}_1, \mathbf{x}_2) = \frac{1}{B} \sum_{m=1}^B \phi_{\nu_m}(\mathbf{x}_1) \phi_{\nu_m}(\mathbf{x}_2). \quad (15)$$

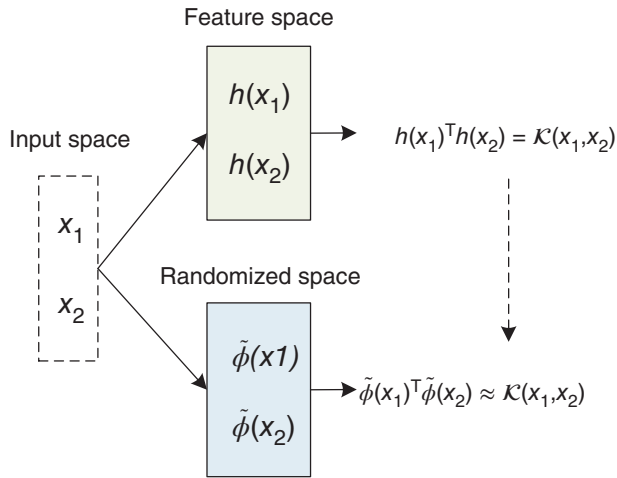


FIGURE 2 | Schematic representation of the kernel approximation process. The original, *fixed* feature mapping is shown in light green, while the randomized space to approximate the kernel evaluation is shown in light blue.

The feature mapping $\tilde{\phi}(\cdot) = [\phi_{v_1}(\cdot), \dots, \phi_{v_B}(\cdot)]$ can now be computed directly. Due to this, all training methods described in the previous section can be applied for optimizing these models, and their universal approximation properties can be analyzed in a similar fashion. In other words, we can apply linear models to approximate nonlinear models that use the original kernel $\mathcal{K}(\cdot, \cdot)$, which in turn indirectly define a nonlinear mapping over a possibly infinite-dimensional space. Standard arguments from probability theory guarantee an exponentially fast decrease of the error of Eq. (15) with respect to Eq. (14) as B increases.⁷ It is important not to mistake the fixed feature mapping $\mathbf{h}(\cdot)$, which uniquely identifies the kernel function, with the randomized space $\tilde{\phi}(\cdot)$, which depends on the sampled vectors $\mathbf{v}_1, \dots, \mathbf{v}_B$. This relation is schematically shown in Figure 2.

A Concrete Example: Random Fourier Features

Random Fourier features (RFF), as originally introduced by Rahimi and Recht,⁷ are the most common implementation of random features for kernel approximation. Here, we briefly introduce them and their extensions. RFFs are defined for any shift-invariant kernel, that is, a kernel that depends on its inputs $\mathbf{x}_1$ and $\mathbf{x}_2$ only by their difference $\mathbf{x}_1 - \mathbf{x}_2$. In this case, it can be shown that the kernel admits a representation similar to Eq. (14) on a Fourier basis with complex functions:

$$\mathcal{K}(\mathbf{x}_1 - \mathbf{x}_2) = \mathbb{E}_{\mathbf{v} \sim P(\mathbf{v})} \left[e^{j\mathbf{v}^T \mathbf{x}_1} e^{-j\mathbf{v}^T \mathbf{x}_2} \right], \quad (16)$$

where j is the imaginary number, and the existence of $P(\mathbf{v})$, $\mathbf{v} \in \mathbb{R}^d$ in closed form is guaranteed by Bochner's theorem from harmonic theory. Substituting the exponential with sines and cosines allows the acquisition of real-valued features so that the final feature vector for RFFs is written as:

$$\tilde{\phi}(\mathbf{x}) = \frac{1}{\sqrt{B}} [\cos(\mathbf{v}_1^T \mathbf{x}), \sin(\mathbf{v}_1^T \mathbf{x}), \dots, \cos(\mathbf{v}_B^T \mathbf{x}), \sin(\mathbf{v}_B^T \mathbf{x})]. \quad (17)$$

As a concrete example, consider the well-known Gaussian kernel, which is by far one of the most widely used kernels in practice thanks to its approximation properties. The Gaussian kernel is defined for a parameter $\gamma \in \mathbb{R}$, similar to α_m in Eq. (4):

$$\mathcal{K}(\mathbf{x}_1, \mathbf{x}_2) = \exp \left\{ -\frac{1}{\gamma^2} \|\mathbf{x}_1 - \mathbf{x}_2\|_2^2 \right\}. \quad (18)$$

In this case, the probability density $P(\mathbf{v})$ is simply a Gaussian density with mean 0 and covariance inversely proportional to γ given by $\gamma^{-1} \mathbf{I}$. RFFs have been studied extensively and have also given rise to a large number of approximation results, which can be translated to the RW-FNN case, as reported in the previous section. Of the methods we review here, they are the only one possessing robust, stable software implementations, such as the one in the Python's scikit-learn library,^c and they have been applied successfully to problems outside supervised learning, such as clustering.⁶⁴

Before concluding this section, we review some of the most recent, promising lines of research devoted to random features:

- Vedaldi and Zisserman⁶² investigated how RFF techniques can be extended to 'generalized' Gaussian kernels, wherein the ℓ_2 norm in Eq. (18) is substituted with a generic distance metric $D(\mathbf{x}_1, \mathbf{x}_2)$. If the distance metric has an additive form, that is, it can be decomposed feature-wise, an efficient random approximation to the generalized kernel can be obtained by combining random features for $D(\cdot, \cdot)$ (using the algorithm presented by the authors) with RFFs features over the kernel. This allows extremely good results to be obtained, particularly in computer vision problems.

- In order to further speed up computations, Le et al.⁶⁵ presented the *fastfood* algorithm, which allows RFFs for the Gaussian kernel to be computed in $\mathcal{O}(\text{Blog}(d))$ time instead of $\mathcal{O}(Bd)$ through the use of specially designed matrices. Their work is generalized in the framework presented by Yang et al.⁶⁶ Along the same lines, a state-of-the-art solver for kernel methods on large-scale scenarios is presented in Dai et al.⁶⁷ by combining RFFs with a stochastic evaluation of the overall gradient.
- Instead of first choosing a kernel and then deriving an approximation, recently, Sinha and Duchi showed that we can work with a generic random feature representation to learn a suitable kernel function for a given task.⁶⁸
- Finally, RFFs have also been studied for solving the last two classes of problems previously detailed, namely the unbounded growth of terms in the kernel expansion⁶⁹ and the possible concerns in privacy-critical settings.⁷⁰ Refined error bounds in the former case have been obtained by Lin et al.⁷¹

Drawbacks and Research Directions

Just like RW-FNNs, the basic problem of random features for kernel approximation is that their sampling strategy does not incorporate information from the training set. Yang et al.⁷² summarize the problem as follows: “[...] *the basis functions used by random Fourier features are sampled from a Gaussian distribution that is independent from the training examples. In contrast, the basis functions used by the Nyström method are sampled from the training examples and are therefore data dependent.*” Basically, the results in terms of classification accuracy in this case will depend on the eigenstructure of the kernel matrix: “*In the case of large eigengap, i.e., the first few eigenvalues of the full kernel matrix are much larger than the remaining eigenvalues, the classification performance is mostly determined by the top eigenvectors. Since the Nyström method uses a data dependent sampling method, it is able to discover the subspace spanned by the top eigenvectors using a small number of samples. In contrast, since random Fourier features are drawn from a distribution independent from training data, it may require a large number of samples before it can discover this subspace.*”

One possibility to counter this problem is to draw a large number B of samples and then linearly downproject the resulting feature mapping to a low-

dimensional space via the use of a RP.⁷³ The error bound in this case is investigated by Hamid et al.⁷³ Another, more principled line of research is to make the sampling strategy dependent on the training data or less subject to variance. Yang et al.⁷² introduced “*a rejection procedure that rejects the sample Fourier components when they do not align well with the top eigenfunctions estimated from the sampled data*” as a possible future line of research. Similar approaches can be found in the later work by the same authors⁶⁶ or in the use of quasi-Monte Carlo procedures for choosing the sequence of points.⁷⁴

It is also worth mentioning the work on sparse spectra in the Gaussian Processes (GPs) literature here,⁷⁵ which exploits the idea of approximating the harmonic representation of the covariance function (the GP equivalent of the kernel function) inside a fully Bayesian framework. Due to this, the optimal frequencies can be chosen according to a data-dependent maximization at the cost of the standard increase in computational burden requested by the Bayesian inference procedure.

RECURRENT NETWORKS WITH RANDOM WEIGHTS

Network Architecture for RC

All the algorithms we investigated up to this point are designed for *static* data, in the sense that the order of presentation of patterns to the network does not influence its behavior. However, many real-world problems require the analysis of *temporal* data that exhibit strong temporal dependencies among subsequent patterns. One way to handle them is to input a feedforward network, such as the RW-FNN previously discussed, with a time-delayed embedding of the temporal series so that a memory of the previous instants is artificially forced by providing them as inputs. However, results in terms of accuracy might be suboptimal, and more importantly, there is no practical way of choosing an embedding dimension of the input. An alternative, more powerful choice is to consider recurrent NNs (RNNs), wherein connections between internal neurons create a temporal processing, making the network a highly nonlinear, flexible dynamic system. RNNs have also achieved remarkable breakthroughs in the last few years with the use of multilayered architectures, gated neuronal units,⁷⁶ efficient second-order optimization routines,⁷⁷ and so on. Still, the training of fully adaptable RNNs requires the flow of error's gradient information throughout temporal instants, increasing the likelihood of vanishing or exploding gradients

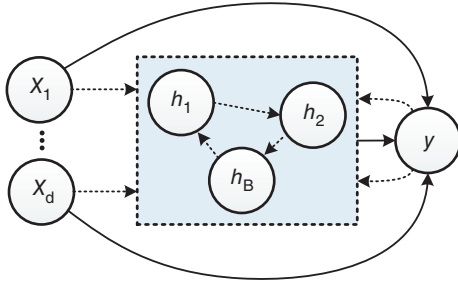


FIGURE 3 | Depiction of an RC architecture with one output. Fixed connections are shown as dashed lines, whilst trainable connections are shown as solid lines. The reservoir is highlighted with a light blue background.

and leading to possibly unstable network behaviors. Even with the diffusion of powerful software libraries allowing automatic differentiation of a cost function over time, the adaptation of RNNs remains a challenging and demanding problem.

For tasks that do not require an extremely long memory of its inputs, an alternative to a fully adaptable RNN is obtained by extending the RW-FNN idea, allowing for a *recurrent* layer of fixed, randomly generated nonlinearities, followed by an adaptable linear layer in the output. This idea, schematically shown in Figure 3, is known in the literature as reservoir computing (RC),⁶ and the two layers are denoted as ‘reservoir’ and ‘readout,’ respectively. There are many flavors of algorithms in the RC field, which are briefly summarized in the next section. Here, we follow the notation and formalism of the so-called echo state network (ESN), which is common in the machine-learning literature.

Denote by $\mathbf{x}[n] \in \mathbb{R}^d$ the d -dimensional input of the ESN at the n th time instant, where we now use the square brackets to highlight the time dependency. Similar to before, the input is fed to a B -dimensional reservoir, whose internal state $\mathbf{h}[n-1] \in \mathbb{R}^B$ is updated according to the state equation:

$$\mathbf{h}[n] = h(\mathbf{W}_i^r \mathbf{x}[n] + \mathbf{W}_r^r \mathbf{h}[n-1] + \mathbf{w}_o^r y[n-1]), \quad (19)$$

where $\mathbf{W}_i^r \in \mathbb{R}^{B \times d}$, $\mathbf{W}_r^r \in \mathbb{R}^{B \times B}$, and $\mathbf{w}_o^r \in \mathbb{R}^B$ are randomly generated matrices and vectors; $h(\cdot)$ is a suitably defined nonlinear function (e.g., a sigmoid); and $y[n-1] \in \mathbb{R}$ is the previous scalar output of the network. To increase stability, it is possible to add a small uniform noise term to the state update before computing the nonlinear transformation $h(\cdot)$.⁷⁸ In the second step, the current output is updated according to:

$$y[n] = (\mathbf{w}_i^o)^T \mathbf{x}[n] + (\mathbf{w}_r^o)^T \mathbf{h}[n], \quad (20)$$

where $\mathbf{w}_i^o \in \mathbb{R}^{N_i}$, $\mathbf{w}_r^o \in \mathbb{R}^{N_r}$ are adapted based on the training data. An alternative update equation with respect to Eq. (19) is to have a reservoir that performs a leaky integration of its state (corresponding in this case to a moving average) for a given parameter $\alpha \in [0, 1]$:

$$\mathbf{h}[n] = \alpha \mathbf{h}[n-1] + (1-\alpha)h(\mathbf{W}_i^r \mathbf{x}[n] + \mathbf{W}_r^r \mathbf{h}[n-1] + \mathbf{w}_o^r y[n-1]). \quad (21)$$

The update in Eq. (21) is the most common variation of leaky ESNs as it only introduces one additional parameter to be tuned, but other choices (e.g., by inserting the leak factor inside the state update) are possible.⁷⁹

ESNs, and RC architectures in general, have achieved remarkable success in many applications, particularly when the problem at hand does not require very long memory processing. Representative examples include short-term load forecasting,¹⁴ grammatical inference,⁸⁰ stock price prediction,⁸¹ speech recognition,⁸² robotic control,⁸³ and acoustic modeling,⁸⁴ among others. Lukoševičius and Jaeger⁶ provide a complete overview of the field up to 2009. A more recent overview of the current trends in RC (some of which are detailed below) is instead provided by Goudarzi and Teuscher.⁸⁵

Training RC Networks

Due to the separation between reservoir and readout, during the training phase, the internal states of the ESN can be computed for each element of the training sequence by using the desired output in Eq. (19) in place of the ESN output, a process known as ‘state harvesting.’ Once this is done, the training is again reduced to a standard linear regression over the concatenation of current input and internal states, allowing for the extremely fast training of a recurrent network architecture as an alternative to a full adaptation. Due to this, all training methods introduced for RW-FNN can also be immediately applied here, including training the network with sparse penalties,⁸⁶ the hinge loss,⁸⁷ and semisupervised formulations.⁸⁸

Historical Overview of RC Classes

The ESN as described in the previous section was formally introduced in 2001 in a technical report by Herbert Jaeger,⁸⁹ and it was designed explicitly to provide a simple, fast alternative to full adaptation of an RNN for machine-learning and signal-processing tasks. In this sense, the neurons in the reservoir act as

dynamic (random) feature extractors that create several ‘echoes’ of the input sequence to be used for classification/regression purposes. For this reason, most of the literature on ESNs (a selection of which is reviewed in the next sections) has focused on creating features that are as useful as possible from a classification point of view. Along this line, some intriguing notions connecting the reservoir (seen as a randomized feature extractor) to infinite NNs and kernel methods are explored by Hermans and Schrauwen.⁹⁰

A very similar algorithm was derived a few years later by Steil,^{91,92} under the name Backpropagation-Decorrelation (BPDC). While the BPDC is slightly more complex than the ESN described previously, it provides an interesting theoretical motivation for having a fixed reservoir. The BPDC algorithm was derived by analyzing the runtime behavior of a (at that time) state-of-the-art algorithm for RNN training, known as Atiya-Parlos.⁹³ The analysis, conducted by Schiller and Steil,⁹⁴ revealed how the algorithm results in a rapidly adapting output layer for the network and a slowly adapting recurrent layer, which also presents strong couplings in some cases. Based on this observation, it makes sense to directly avoid the adaptation of the latter, resulting in a simplified linear problem and in the BPDC algorithm itself.

A similar idea to ESNs was proposed in 2002 by Maass et al., under the name liquid state machine (LSM),⁹⁵ from a completely different point of view, namely as a model for computational neuroscience. LSMs were put forth as a mathematical tool for analyzing the properties of neural microcircuits, showing that layers of neurons can learn to ‘act’ for different tasks on the information coming from a large (fixed) reservoir of neurons, exploiting the high dimensionality of the feature mapping, and thereby providing “*a computational model that does not require convergence to stable internal states or attractors*” of the recurrent neural part,⁹⁵ in contrast to the previous literature. The study of LSMs provides a direct link between artificial NNs as used in machine learning and neuroscience, particularly with respect to the study of random populations of neurons and their biological plausibility (e.g., by providing a connection to the decades-long investigation of P.F. Dominey and colleagues⁹⁶). In the last decade, LSMs have been a fundamental tool for the understanding of specific neural models and their computational properties.^{97,98}

Differentiated from ESNs, LSMs are designed in order to be biologically plausible, and for this reason, they employ more sophisticated spiking models

of neurons, and more importantly, the connectivity of the reservoir (called the ‘liquid’ in the LSM) follows biological principles of organization. Nonetheless, all the models discussed here are fundamentally equivalent, and in 2007, they were unified in the RC framework by the work of Verstraeten et al.⁹⁹ Recently, an evolution of the idea of ESNs with spiking neurons was proposed under the name NeuCube, in which temporal information for multiple time series is mapped to different parts of a 3D reservoir based on their spatiotemporal properties.^{100,101}

More generally, the idea of randomly selecting weights in RNNs was first explored by Schmidhuber and Hochreiter in a 1996 report,¹⁰² where they showed how complete guessing was outperforming current state-of-the-art training methods in many simple problems. Here, we also briefly mention the ‘random NN’ model, analyzed by Gelenbe in a number of works during the 90s.^{103,104} In this network, randomness is introduced in the rate of firing among neurons, and the weights representing these rates are adapted by repeatedly solving two sets of linear and nonlinear equations for each input/output pair presented to the network. Due to this, the model does not fall in the unified presentation given in this overview, although several of its ideas could be exploited in connection to RC networks. A survey on the applications of these models was conducted by Bakircioğlu and Koçak¹⁰⁵

THEORETICAL PROPERTIES OF RC NETWORKS

The reservoir is the central component of RC networks, and its presence is the major difference with respect to feedforward models. Although its parameters are not adapted in a supervised fashion, analyzing its dynamic behavior is essential to understanding the capabilities of RC networks. Intuitively, if we are able to avoid ‘unstable’ reservoirs, the universal approximation properties of RW-FNNs apply immediately; this is the topic of the first part of this section. In the rest of the section, we investigate some additional aspects related to the reservoir in terms of topology, computational power, and memory.

The Echo State Property

In RW-FNNs, connections in the first layer are generally extracted from a fixed probability distribution, typically a uniform distribution in $[-1, +1]$, although (as we have seen) this is not enough to guarantee its accuracy for any possible initialization. However, this

approach is not sufficient for ESNs: reservoirs with random weights can result in dynamic systems that oscillate or, even worse, present chaotic behaviors. Clearly, the features extracted by these reservoirs would be of limited use in terms of discriminatory power. Basically, we should be able to guarantee that the effects of any given input and state vanish in a finite number of time instants without persisting indefinitely or becoming amplified exponentially. This property of reservoirs was called the echo state property (ESP) by Jaeger,⁸⁹ while a similar property was called the ‘input separation property’ in the LSM literature. It can be shown that, under these conditions, ESNs can approximate any time series with fading memory to the desired level of accuracy (whereas it cannot be used for tasks requiring unbounded memory).⁹⁵

In practice, the ESP can be guaranteed *almost* always by rescaling the matrix $\mathbf{W}_r^r$ so that its spectral radius $\rho(\mathbf{W}_r^r)$ (defined as the maximum eigenvalue in absolute terms) is less than 1. This rule originates from the work of Jaeger, which also proposed to have sparse connectivity in $\mathbf{W}_r^r$ so as to create heterogeneous clusters of features.⁸⁹ Together, these two rules represent the most common way of initializing the ESN architecture.

Nonetheless, this criterion is only heuristic and does not exactly guarantee the ESP. Jaeger⁸⁹ showed that the ESP is necessarily violated if $\rho(\mathbf{W}_r^r) > 1$ for networks with *tanh* nonlinearities and if the input space contains the zero sequence, but the criterion does not relate to ESNs driven by a specific input sequence, and the ESP might be lost (theoretically) even for $\rho(\mathbf{W}_r^r) \leq 1$. A sufficient condition in terms of the largest singular value was also obtained by Jaeger, but it is too restrictive to be useful in practice. Two equivalent (tighter) sufficient conditions were obtained by Buehner and Young in 2006 and by Yildiz et al. in 2012.^{106,107} Taking into account the random initialization of the reservoir, Zhang et al.¹⁰⁸ instead exploited tools from random matrix theory to analyze the behavior of the network, showing that (asymptotically) the empirical criterion $\rho(\mathbf{W}_r^r) \leq 1$ is in fact justified. Some important results relating the ESP to the nature of the input signal were obtained in 2013 by Manjunath and Jaeger.¹⁰⁹

We emphasize that the ESP property is not the only possibility to design stable reservoirs in a principled way. Ozturk et al.¹¹⁰ proposed a metric based on information-theoretic arguments by analyzing the linearized dynamic of the reservoir. To date, this remains the most common alternative to the *a priori* selection of a desired spectral radius. In practice, both the input matrix and the feedback matrix have

a great impact on the prediction performance. Note that the criterion $\rho(\mathbf{W}_r^r) \leq 1$ can be regarded as a constraint on the distribution of the random weights of the feedback matrix, which can be characterized in the sense of the ℓ_1 norm by the scope of random weights. Thus, a proper estimate on the scope setting for both the input and feedback matrices is critical to guarantee a success of time series modeling (a typical data-mining task).

Design of the Reservoir

Since the training of the readout is formulated as a linear problem, most of the relevant literature can also be applied for this class of networks. As an example, ℓ_1 minimization to achieve sparse readouts was investigated independently in the context of ESNs by Ceperic and Baric¹³ and Bianchi et al.,¹⁴ with some similar results obtained in a previous study by Butcher et al.⁸⁶ Additionally, it is possible to consider advanced optimization strategies,^{111,112} support vector algorithms,⁸⁷ and many other variations (see Section 7 of Lukoševičius and Jaeger for an overview⁶).

In a more general sense, however, the isolation of the recurrent part of the network allows for a lot of freedom in its design, both by varying its initialization strategy (in terms of connectivity) and by adding unsupervised strategies for its optimization.^{113,114} In the latter case, submodular reservoirs with lateral inhibition¹¹³ and intrinsic plasticity (IP) are two state-of-the-art unsupervised strategies with a biological inspiration.¹¹⁴ Lateral inhibition is a biological mechanism in which the activity of a neuron might inhibit the response of another connected neuron. A way to realize this in ESNs is to have multiple (smaller) reservoirs, with pairwise negative connections to inhibit their neighbors.¹¹³ As an example, consider two reservoirs described by matrices $\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^{\frac{B}{2} \times \frac{B}{2}}$, where each reservoir is now constituted by $\frac{B}{2}$ neurons. An overall reservoir with lateral inhibition can be constructed as:

$$\mathbf{W} = \begin{bmatrix} \mathbf{W}_1 & -\mathcal{P}\mathbf{I} \\ -\mathcal{P}\mathbf{I} & \mathbf{W}_2 \end{bmatrix}, \quad (22)$$

where $\mathcal{P}$ is an operator that predicts the new subreservoir states in the absence of lateral inhibition, and we suppose that both inhibitory connections have the same strength.¹¹³ Despite its simplicity, this scheme can have a significant impact on prediction accuracy.¹¹³ More generally, clustered architectures for the reservoir have been studied extensively as they

can also provide improvements on the overall stability of the system.¹¹⁵

IP, proposed by Steil¹¹⁶ in 2007 and extended by Schrauwen et al.,¹¹⁴ is another biologically inspired rule that can be implemented easily. Practically, it allows for an increase in accuracy and a possible decrease in variance due to the random initialization. Suppose that the original reservoir's nonlinearity $b(s)$ is parameterized with two free adaptable values $a, b \in \mathbb{R}$ as $b_{ip} = b(as + b)$. The IP rule allows the fine tuning of these two parameters in an online fashion so that the distribution of the reservoirs' states approaches a predefined distribution, for example, Gaussian. Simple update rules can be found in closed form for most nonlinearities used in practice.^{114,116}

Another possibility for improving performance is to consider layered architectures in which the outputs of one layer are provided as inputs to a subsequent layer. Triefenbach et al.⁸⁴ investigated this idea for acoustic modeling in which every layer of the network is trained to distinguish between acoustic entities of increasing temporal duration. Alternative approaches, comprising multiple reservoirs connected to a *single* readout and/or between them, are explored in two recent papers by Malik et al.¹¹⁷ and Galicchio and Micheli.¹¹⁸ Despite the good results obtained, multilayered models are uncommon in ESNs, and in fact, they represent one of the most interesting open questions as they might be able to process events at different time resolutions. An alternative line of research has focused on the construction of simpler, *deterministic* reservoirs. Specifically, Rodan and Tiño^{119,120} showed that cyclic reservoirs with possibly highly constrained weights can perform almost as well as large, random reservoirs. Similar insights are obtained by Strauss et al.¹²¹ Together, these results raise some important questions, namely: are the random 'echoes' produced by ESNs actually helpful, or can we discard randomness and focus on the algebraic properties of the matrices? Furthermore, in this case, is this true for any possible class of problems?

Recently, Appeltant et al.¹²² showed good performance of a 'virtual' reservoir composed of a single node receiving time-delayed versions of its input. This scheme is particularly interesting as the algorithm admits a very fast electronic implementation. Later, fully optoelectronic implementations of ESN schemes were independently presented in Paquot et al.¹²³ and Larger et al.,¹²⁴ paving the way to ultrafast implementations of RC using nonconventional computing technologies.

Computational Power and Memory in RC Architectures

Apart from the initialization strategy of the reservoir and its spectral radius, another fundamental parameter in the design of ESNs is the range in which the input connections $\mathbf{W}_i^r$ are drawn. Basically, larger connections drive the reservoir in more nonlinear regions for commonly used nonlinear functions. Together, these parameters determine the computational regime in which the ESN is operating, both in terms of computational power and memory capacity.^{125,126} One important aspect that we briefly mention here is the concept of 'edge of instability' (or 'edge of chaos'); namely, a large body of literature suggests that ESNs, like other recurrent architectures and even cortical circuits, have the highest computational power when they operate near an instable regime.^{127,128} This notion is of paramount importance as it provides a link between RC, neuroscience, and the theory of complex systems. Open questions abound, particularly in order to determine *when* such processing power is required and, in this case, how we can detect the onset of criticality in order to achieve this aim.^{129–131}

CONCLUSIONS

Randomly assigning a subset of parameters in a learner model is an efficient approach for training NNs, which provides an effective solution for large-scale data-mining problems. Methods exploiting random mappings, such as RVFL networks, Random Kitchen Sinks, and FastFood, commonly share a two-step process, that is, random mapping using some weights (must be carefully scoped for the universal approximation property) followed by resolving a linear model. Due to the formulation of optimization problems in the parameter space, all solutions can be obtained by leveraging over algebra routines and well-developed techniques for linear regression and classification. Throughout this overview, we have seen how this idea can be applied to feedforward NNs, RNNs, and for approximating kernel methods defined over possibly infinite-dimensional feature spaces. A wealth of algorithms can be cast under a rather general framework, providing good tradeoffs in terms of accuracy, training time, extensibility, and so on. Nonetheless, advancement in these models has been penalized by a fragmentation of the literature over different fields. In this paper, we provided a comprehensive overview of all of them, showing their history, strengths, weaknesses, and—most importantly—open problems. As we have seen, many

fundamental questions remain to be further explored in order to bridge an existing gap between purely randomized architectures and fully adaptable networks. A particularly daunting problem is to develop some practical methods for the design of sampling distributions in a data-dependent fashion, which will play a key role in dealing with streaming data-mining problems.

There are some interesting and primary questions on the randomized approaches; for instance, is randomness beneficial to general data-mining problems, or is there a way of constructing informative, functional, and cheap feature spaces without randomization techniques that can perform favorably?^{119,120} Additional insights might be obtained by connecting the theory presented here to the broad field of randomness and stability in learning

theory,¹³² which investigates how randomness in an algorithm affects the resulting error bounds. In the end, we hope that this overview will attract more researchers to pay attention to the use of randomness in the field of machine learning and data mining, fostering innovative methods and the exchange of key results among communities.

NOTES

^a <http://spark.apache.org/docs/latest/mllib-linear-methods.html#regression>

^b We use the same notation as before, as the two mappings share the same purpose.

^c http://scikit-learn.org/stable/modules/kernel_approximation.html

FURTHER READING

For readers interested in the field of randomization for NNs and kernel functions, we recommend the recent special issue on the topic published in *Information Sciences*. Specifically, the special issue contains an informative editorial by Wang¹³³ and a survey paper by Zhang and Suganthan.¹³⁴ Apart from the algorithms detailed here, the survey covers additional methods that venture outside the definition of randomization we considered, such as probabilistic NNs,¹³⁵ ensemble methods,¹³⁶ dropout regularization,¹³⁷ and others. As such, it provides an excellent alternative entry point into the growing literature of the field.

REFERENCES

1. Glorot X, Bengio Y. In: *13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, Sardinia, Italy, May 13–15, 2010, 249–256.
2. Hofmann T, Schölkopf B, Smola A. Kernel methods in machine learning. *Ann Stat* 2008, 36:1171–1220.
3. Schmidhuber J. Deep learning in neural networks: An overview. *Neural Netw* 2015, 61:85–117.
4. Kotsiantis S. Supervised machine learning: A review of classification techniques. *Informatica* 2007, 31:249–268.
5. Pao Y-H, Park G-H, Sobajic D. Learning and generalization characteristics of the random vector functional-link net. *Neurocomputing* 1994, 6:163–180.
6. Lukoševicius M, Jaeger H. Reservoir computing approaches to recurrent neural network training. *Comput Sci Rev* 2009, 3:127–149.
7. Rahimi A, Recht B. In: *Advances in Neural Information Processing Systems*. MIT Press; Vancouver, Canada, December 3–6, 2007:1177–1184.
8. Cover T. Geometrical and statistical properties of systems of linear inequalities with applications in pattern recognition. *IEEE Trans Electron Comput* 1965, EC-14:326–334.
9. Rahimi A, Recht B. In: *Advances in Neural Information Processing Systems*. MIT Press; Vancouver, Canada, December 6–9, 2009:1313–1320.
10. Stigler S. Gauss and the invention of least squares. *Ann Stat* 1981, 9:465–474.
11. Bach F, Jenatton R, Mairal J, Obozinski G. Optimization with sparsity-inducing penalties. *Found Trends Mach Learn* 2012, 4:1–106.
12. Cao F, Tan Y, Cai M. Sparse algorithms of random weight networks and applications. *Expert Syst Appl* 2014, 41:2457–2462.
13. Ceperic V, Baric A. In: *2014 UKSim-AMSS 16th International Conference on Computer Modelling and Simulation (UKSim)*, Cambridge, MA, US, March 26–28, 2014, 26–31.
14. Bianchi F, Scardapane S, Uncini A, Rizzi A, Sadeghian A. Prediction of telephone calls load using Echo State Network with exogenous variables. *Neural Netw* 2015, 71:204–213.

15. Lu Z, May A, Liu K, Garakani A, Guo D, Bellet A, Fan L, Collins M, Kingsbury B, Picheny M, et al. How to scale up kernel methods to be as good as deep neural nets. arXiv preprint arXiv:1411.4000. 2014.
16. Saxe A, Koh P, Chen Z, Bhand M, Suresh B, Ng A. In: *28th International Conference on Machine Learning (ICML)*, Bellevue, WA, US, June 28–July 2, 2011, 1089–1096.
17. Bingham E, Mannila H. In: *Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York, NY: ACM; San Francisco, CA, US, August 26–29, 2001, 245–250.
18. Fradkin D, Madigan D. In: *Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York, NY: ACM; Washington, DC, US, August 24–27, 2003, 517–522.
19. Lowe D, Broomhead D. Multivariable functional interpolation and adaptive networks. *Complex Syst* 1988, 2:321–355.
20. Igel'nik B, Pao Y-H, LeClair S, Shen C. The ensemble approach to neural network learning and generalization. *IEEE Trans Neural Netw* 1999, 10:19–30.
21. Jarrett K, Kavukcuoglu K, Ranzato M, LeCun Y. In: *2009 I.E. 12th International Conference on Computer Vision (ICCV)*. Piscataway, NJ: IEEE; Kyoto, Japan, September 27–October 4, 2009, 2146–2153.
22. Chu C, Kim S, Lin Y-A, Yu Y, Bradski G, Ng A, Olukotun K. In: *Advances in Neural Information Processing Systems*. Cambridge, MA: MIT Press; Vancouver, Canada, December 3–6, 2007, 281–288.
23. Agullo E, Augonnet C, Dongarra J, Faverge M, Ltaief H, Thibault S, Tomov S. In: *2011 I.E. International Parallel and Distributed Processing Symposium (IPDPS)*. Piscataway, NJ: IEEE; Anchorage, Alaska, US, May 16–20, 2011, 932–943.
24. Scardapane S, Wang D, Panella M, Uncini A. Distributed learning for random vector functional-link networks. *Inform Sci* 2015, 301:271–284.
25. Leung C, Young G, Sum J, Kan W-K. On the regularization of forgetting recursive least square. *IEEE Trans Neural Netw* 1999, 10:1482–1486.
26. Yuan G-X, Ho C-H, Lin C-J. Recent advances of large-scale linear classification. *Proc IEEE* 2012, 100:2584–2603.
27. Cao F, Ye H, Wang D. A probabilistic learning algorithm for robust modeling using neural networks with random weights. *Inform Sci* 2015, 313:62–78.
28. Alhamdoosh M, Wang D. Fast decorrelated neural network ensembles with random weights. *Inform Sci* 2014, 264:104–117.
29. Seber G, Lee A. *Linear Regression Analysis*. Hoboken, NJ: John Wiley & Sons; 2012.
30. Rosenblatt F. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychol Rev* 1958, 65:386–408.
31. Widrow B, Lehr M. 30 years of adaptive neural networks: Perceptron, madaline, and backpropagation. *Proc IEEE* 1990, 78:1415–1442.
32. Gallant S, Smith D. In: *1987 I.E. International Conference on Neural Networks (IJCNN)*, San Diego, CA, US, June 21–24, 1987.
33. Minsky M, Papert S. *Perceptrons*. Cambridge, MA: MIT Press; 1988.
34. Pao Y, Takefji Y. Functional-link net computing: Theory, system architecture, and functionalities. *IEEE Comput J* 1992, 25:76–79.
35. Igel'nik B, Pao Y-H. Stochastic choice of basis functions in adaptive function approximation and the functional-link net. *IEEE Trans Neural Netw* 1995, 6:1320–1329.
36. Park G-H, Pao Y-H. Unconstrained word-based approach for off-line script recognition using density-based random-vector functional-link net. *Neurocomputing* 2000, 31:45–65.
37. Husmeier D, Taylor J. Neural networks for predicting conditional probability densities: Improved training scheme combining EM and RVFL. *Neural Netw* 1998, 11:89–116.
38. Martinez-Villena J, Rosado-Muñoz A, Soria-Olivas E. Hardware implementation methods in random vector functional-link networks. *Appl Intell* 2014, 41:184–195.
39. Zhang L, Suganthan P. A comprehensive evaluation of random vector functional link networks. *Inform Sci* 2016, 367–368:1094–1105.
40. Scardapane S, Comminiello D, Scarpiniti M, Uncini A. A semi-supervised random vector functional-link network based on the transductive framework. *Inform Sci* 2016, 364:156–166.
41. Cecotti H. In: *2016 International Joint Conference on Neural Networks (IJCNN)*. Piscataway, NJ: IEEE; Vancouver, Canada, July 24–29, 2016, 3628–3633.
42. Looney C. Radial basis functional link nets and fuzzy reasoning. *Neurocomputing* 2002, 48:489–509.
43. Schmidt W, Kraaijveld M, Duin R. In: *11th IAPR International Conference on Pattern Recognition (ICPR)*. Piscataway, NJ: IEEE; The Hague, Netherlands, August 30–September 3, 1992, 1–4.
44. Duin R. Superlearning and neural network magic. *Pattern Recogn Lett* 1994, 15:215–217.
45. Widrow B. Reply to the comments on the “No-Prop” algorithm. *Neural Netw* 2013, 48:204.
46. Widrow B, Greenblatt A, Kim Y, Park D. The no-prop algorithm: A new learning algorithm for multilayer neural networks. *Neural Netw* 2013, 37:182–188.
47. Tapsos J, van Schaik A. Learning the pseudoinverse solution to network weights. *Neural Netw* 2013, 45:94.
48. Li J-Y, Chow W, Igel'nik B, Pao Y-H. Comments on “Stochastic choice of basis functions in adaptive function approximation and the functional-link net”. *IEEE Trans Neural Netw* 1997, 8:452–454.

49. Barron A. Universal approximation bounds for superpositions of a sigmoidal function. *IEEE Trans Inf Theory* 1993, 39:930–945.
50. Gorban A, Tyukin I, Prokhorov D, Sofeikov K. Approximation with random bases: Pro et contra. *Inform Sci* 2016, 364–365:129–145.
51. Principe J, Chen B. Universal approximation with convex optimization: Gimmick or reality? *IEEE Comput Intell Mag* 2015, 10:68–77.
52. Li M, Wang D. Insights into randomized algorithms for neural networks: Practical issues and common pitfalls. *Inform Sci* 2017, 382–383:170–178.
53. Rahimi A, Recht B. In: *46th Annual Allerton Conference on Communication, Control, and Computing*. Piscataway, NJ: IEEE; Monticello, IL, US, September 26–28, 2008, 555–561.
54. Rudi A, Camoriano R, Rosasco L. Generalization properties of learning with random features. arXiv preprint arXiv:1602.04474, 2016.
55. Cortes C, Vapnik V. Support-vector networks. *Mach Learn* 1995, 20:273–297.
56. Van Vaerenbergh S, Lázaro-Gredilla M, Santamara I. Kernel recursive least-squares tracker for time-varying regression. *IEEE Trans Neural Netw Learn Syst* 2012, 23:1313–1326.
57. Wu X, Zhu X, Wu G-Q, Ding W. Data mining with big data. *IEEE Trans Knowl Data Eng* 2014, 26:97–107.
58. Achlioptas D, McSherry F, Schölkopf B. In: *Advances in Neural Information Processing Systems*, vol 1. Cambridge, MA: MIT Press; Vancouver, Canada, December 9–14, 2002, 335–343.
59. Drineas P, Mahoney M. On the Nystrom method for approximating a Gram matrix for improved kernel-based learning. *J Mach Learn Res* 2005, 6:2153–2175.
60. Williams C, Seeger M. In: *Advances in Neural Information Processing Systems*. Cambridge, MA: MIT Press; Vancouver, Canada, December 3–8, 2001, 682–688.
61. Li F, Ionescu C, Sminchisescu C. In: *Pattern Recognition*. New York, NY: Springer; 2010, 262–271.
62. Vedaldi A, Zisserman A. Efficient additive kernels via explicit feature maps. *IEEE Trans Pattern Anal Mach Intell* 2012, 34:480–492.
63. Kar P, Karnick H. In: *15th International Conference on Artificial Intelligence and Statistics (AISTATS)*, La Palma, Canary Islands, April 21–23, 2012.
64. Chitta R, Jin R, Jain A. In: *2012 I.E. 12th International Conference on Data Mining (ICDM)*. Piscataway, NJ: IEEE; Brussels, Belgium, December 10–13, 2012, 161–170.
65. Le Q, Sarlós T, Smola A. In: *30th International Conference on Machine Learning (ICML)*, Atlanta, GA, US, June 16–21, 2013, 244–252.
66. Yang Z, Smola A, Song L, Wilson A. In: *18th International Conference on Artificial Intelligence and Statistics (AISTATS)*, San Diego, CA, US, May 9–12, 2015.
67. Dai B, Xie B, He N, Liang Y, Raj A, Balcan M-FF, Song L. In: *Advances in Neural Information Processing Systems*, Montreal, Canada, December 8–13, 2014, 3041–3049.
68. Sinha A, Duchi JC. In: *Advances in Neural Information Processing Systems*, Barcelona, Spain, December 5–10, 2016, 1298–1306.
69. Lu J, Hoi S, Wang J, Zhao P, Liu Z-Y. Large scale online kernel learning. *J Mach Learn Res* 2016, 17:1–43.
70. Chaudhuri K, Monteleoni C, Sarwate A. Differentially private empirical risk minimization. *J Mach Learn Res* 2009, 12:1069–1109.
71. Lin M, Weng S, Zhang C. On the sample complexity of random Fourier features for online learning: How many random Fourier features do we need? *ACM Trans Knowl Discov Data* 2014, 9:39:1–39:27.
72. Yang JC, Li Y-F, Mahdavi M, Jin R, Zhou Z-H. In: *Advances in Neural Information Processing Systems*, Lake Tahoe, NV, US, December 3–8, 2012, 476–484.
73. Hamid R, Xiao Y, Gittens A, Decoste D. Compact random feature maps. arXiv preprint arXiv:1312.4626, 2013.
74. Yang J, Sindhwani V, Avron H, Mahoney M. In: *31st International Conference on Machine Learning (ICML)*, Beijing, China, June 21–26, 2014, 485–493.
75. Lázaro-Gredilla M, Quiñero-Candela J, Rasmussen CE, Figueiras-Vidal AR. Sparse spectrum Gaussian process regression. *J Mach Learn Res* 2010, 11:1865–1881.
76. Sutskever I, Vinyals O, Le Q. In *Advances in Neural Information Processing Systems*, Montreal, Canada, December 8–13, 2014, 3104–3112.
77. Martens J, Sutskever I. In: *Neural Networks: Tricks of the Trade, Reloaded*. Berlin: Springer; 2012, 479–535 (Chapter 20).
78. Jaeger H. In: *Advances in Neural Information Processing Systems*, Vancouver, Canada, December 9–14, 2002, 593–600.
79. Jaeger H, Lukoševicius M, Popovici D, Siewert U. Optimization and applications of echo state networks with leaky-integrator neurons. *Neural Netw* 2007, 20:335–352.
80. Tong M, Bickett A, Christiansen E, Cottrell G. Learning grammatical structure with echo state networks. *Neural Netw* 2007, 20:424–432.
81. Lin X, Yang Z, Song Y. Short-term stock price prediction based on echo state networks. *Expert Syst Appl* 2009, 36:7313–7317.
82. Skowronski MD, Harris JG. Automatic speech recognition using a predictive echo state network classifier. *Neural Netw* 2007, 20:414–423.

83. Aislan Antonelo E, Schrauwen B. On learning navigation behaviors for small mobile robots with reservoir computing architectures. *IEEE Trans Neural Netw Learn Syst* 2015, 26:763–780.
84. Triefenbach F, Jalalvand A, Demuynck K, Martens J-P. Acoustic modeling with hierarchical reservoirs. *IEEE Trans Audio Speech Lang Process* 2013, 21:2439–2450.
85. Goudarzi A, Teuscher C. In: *3rd ACM International Conference on Nanoscale Computing and Communication*. New York, NY: ACM; New York, NY, US, September 28–30, 2016, 13.
86. Butcher J, Day C, Haycock P, Verstraeten D, Schrauwen B. In: *2010 I.E. International Workshop on Machine Learning for Signal Processing (MLSP)*, Kittilä, Finland, August 29–September 1, 2010, 250–255.
87. Li D, Han M, Wang J. Chaotic time series prediction based on a novel robust echo state network. *IEEE Trans Neural Netw Learn Syst* 2012, 23:787–799.
88. Scardapane S, Uncini A. Semi-supervised echo state networks for audio classification. *Cogn Comput* 2016. In press.
89. Jaeger H. German National Research Center for Information Technology GMD Technical Report 148, 2001.
90. Hermans M, Schrauwen B. Recurrent kernel machines: Computing with infinite echo state networks. *Neural Comput* 2012, 24:104–133.
91. Steil J. In: *2004 I.E. International Joint Conference on Neural Networks (IJCNN)*, vol 2. Piscataway, NJ: IEEE; Budapest, Hungary, July 25–29, 2004, 843–848.
92. Steil J. Online stability of backpropagation-decorrelation recurrent learning. *Neurocomputing* 2006, 69:642–650.
93. Atiya A, Parlos A. New results on recurrent network training: Unifying the algorithms and accelerating convergence. *IEEE Trans Neural Netw* 2000, 11:697–709.
94. Schiller U, Steil J. Analyzing the weight dynamics of recurrent learning algorithms. *Neurocomputing* 2005, 63:5–23.
95. Maass W, Natschläger T, Markram H. Real-time computing without stable states: A new framework for neural computation based on perturbations. *Neural Comput* 2002, 14:2531–2560.
96. Dominey P. Complex sensory-motor sequence learning based on recurrent state representation and reinforcement learning. *Biol Cybern* 1995, 73:265–274.
97. Yamazaki T, Tanaka S. The cerebellum as a liquid state machine. *Neural Netw* 2007, 20:290–297.
98. Hinaut X, Dominey P. Real-time parallel processing of grammatical structure in the fronto-striatal system: A recurrent network simulation study using reservoir computing. *PLoS One* 2013, 8:e52946.
99. Verstraeten D, Schrauwen B, D’Haene M, Stroobandt D. An experimental unification of reservoir computing methods. *Neural Netw* 2007, 20:391.
100. Kasabov N, Capecci E. Spiking neural network methodology for modelling, classification and understanding of EEG spatio-temporal data measuring cognitive processes. *Inform Sci* 2015, 294:565–575.
101. Tu E, Kasabov N, Yang J. Mapping temporal variables into the NeuCube for improved pattern recognition, predictive modeling, and understanding of stream data. *IEEE Trans Neural Netw Learn Syst*. In press.
102. Schmidhuber J, Hochreiter S. Guessing can outperform many long time lag algorithms. Technical Note IDSIA-19-96, 1996.
103. Gelenbe E. Stability of the random neural network model. *Neural Comput* 1990, 2:239–247.
104. Gelenbe E. Learning in the recurrent random neural network. *Neural Comput* 1993, 5:154–164.
105. Bakircioğlu H, Koçak T. Survey of random neural network applications. *Eur J Oper Res* 2000, 126:319–330.
106. Buehner M, Young P. A tighter bound for the echo state property. *IEEE Trans Neural Netw* 2006, 17:820–824.
107. Yildiz I, Jaeger H, Kiebel S. Re-visiting the echo state property. *Neural Netw* 2012, 35:1–9.
108. Zhang B, Miller D, Wang Y. Nonlinear system modeling with random matrices: Echo state networks revisited. *IEEE Trans Neural Netw Learn Syst* 2012, 23:175–182.
109. Manjunath G, Jaeger H. Echo state property linked to an input: Exploring a fundamental characteristic of recurrent neural networks. *Neural Comput* 2013, 25:671–696.
110. Ozturk M, Xu D, Principe J. Analysis and design of echo state networks. *Neural Comput* 2007, 19:111–138.
111. Sheng C, Zhao J, Liu Y, Wang W. Prediction for noisy nonlinear time series by echo state network based on dual estimation. *Neurocomputing* 2012, 82:186–195.
112. Scardapane S, Wang D, Panella M. A decentralized training algorithm for echo state networks in distributed big data applications. *Neural Netw* 2016, 78:65–74.
113. Xue Y, Yang L, Haykin S. Decoupled echo state networks with lateral inhibition. *Neural Netw* 2007, 20:365–376.
114. Schrauwen B, Wardermann M, Verstraeten D, Steil J, Stroobandt D. Improving reservoirs using intrinsic plasticity. *Neurocomputing* 2008, 71:1159–1171.
115. Kaiser M, Hilgetag C, Kötter R. Hierarchy and dynamics of neural networks. *Front Neuroinform* 2010, 4:1–3.

116. Steil J. Online reservoir adaptation by intrinsic plasticity for backpropagation-decorrelation and echo state learning. *Neural Netw* 2007, 20:353–364.
117. Malik Z, Hussain A, Wu QJ. Multi-layered echo state machine: A novel architecture and algorithm. *IEEE Trans Cybern*. In press.
118. Galicchio C, Micheli A. In: *ESANN 2016 – European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning*, Bruges, Belgium, April 27–29, 2016, 1–6.
119. Rodan A, Tiño P. Minimum complexity echo state network. *IEEE Trans Neural Netw* 2011, 22:131–144.
120. Rodan A, Tiño P. Simple deterministically constructed cycle reservoirs with regular jumps. *Neural Comput* 2012, 24:1822–1852.
121. Strauss T, Wustlich W, Labahn R. Design strategies for weight matrices of echo state networks. *Neural Comput* 2012, 24:3246–3276.
122. Appeltant L, Soriano M, Van der Sande G, Danckaert J, Massar S, Dambre J, Schrauwen B, Mirasso C, Fischer I. Information processing using a single dynamical node as complex system. *Nat Commun* 2011, 2:1–6.
123. Paquot Y, Duport F, Smerieri A, Dambre J, Schrauwen B, Haelterman M, Massar S. Optoelectronic reservoir computing. *Sci Rep* 2012, 2:1–6.
124. Larger L, Soriano M, Brunner D, Appeltant L, Gutiérrez J, Pesquera L, Mirasso C, Fischer I. Photonic information processing beyond Turing: an optoelectronic implementation of reservoir computing. *Opt Express* 2012, 20:3241–3249.
125. Jaeger H. Short term memory in echo state networks. German National Research Center for Information Technology GMD Technical Report, 2001.
126. Verstraeten D, Dambre J, Dutoit X, Schrauwen B. In: 2010 I.E. *International Joint Conference on Neural Networks (IJCNN)*. Piscataway, NJ: IEEE; Barcelona, Spain, July 18–23, 2010, 1–8.
127. Bertschinger N, Natschläger T. Real-time computation at the edge of chaos in recurrent neural networks. *Neural Comput* 2004, 16:1413–1436.
128. Boedecker J, Obst O, Lizier J, Mayer N, Asada M. Information processing in echo state networks at the edge of chaos. *Theory Biosci* 2012, 131:205–213.
129. Goudarzi A, Stefanovic D. Towards a calculus of echo state networks. *Procedia Comput Sci* 2014, 41:176–181.
130. Livi L, Bianchi F, Alippi C. Determination of the edge of criticality in echo state networks through Fisher information maximization. *IEEE Trans Neural Netw Learn Syst*. In press.
131. Bianchi F, Livi L, Alippi C. Investigating echo state networks dynamics by means of recurrence analysis. *IEEE Trans Neural Netw Learn Syst*. In press.
132. Elisseeff A, Evgeniou T, Pontil M. Stability of randomized learning algorithms. *J Mach Learn Res* 2005, 6:55–79.
133. Wang D. Editorial: Randomized algorithms for training neural networks. *Inform Sci* 2016, 364–365:126–128.
134. Zhang L, Suganthan P. A survey of randomized algorithms for training neural networks. *Inform Sci* 2016, 364–365:146–155.
135. Salakhutdinov R, Hinton G. In: *12th International Conference on Artificial Intelligence and Statistics (AISTATS)*, Clearwater Beach, FL, US, April 16–18, 2009, 448–455.
136. Ren Y, Zhang L, Suganthan P. Ensemble classification and regression – recent developments, applications and future directions. *IEEE Comput Intell Mag* 2016, 11:41–53.
137. Srivastava N, Hinton G, Krizhevsky A, Sutskever I, Salakhutdinov R. Dropout: A simple way to prevent neural networks from overfitting. *J Mach Learn Res* 2014, 15:1929–1958.